

KALKI · TECHNICAL GAP ANALYSIS

The Engineering Translation Layer

The V2.0 Master Build Specification tells its developer what to build. This companion tells them how: the YANTRA engine sub-specification, the canonical data model, the Seeker state contract, the station-gate formula, register-integrity machinery, error states, and the roadmap that sequences them.

Companion to the V2.0 Master Build Specification

Grounded in Strategy Dossier No. 03 · The Visibility Engine

August 2026 · Version 1.0 · **For the build developer**

Contents

- 1. Executive Summary 3**
- 2. How to Read This Document 4**
 - 2.1 Severity model 4
 - 2.2 Citation and artifact conventions 4
- 3. The Gap Register 5**
- 4. Strengths and Their Engineering Obligations 6**
 - 4.1 S-01 · The four-register taxonomy 6
 - 4.2 S-02 · The Dossier as rite of entry 6
 - 4.3 S-03 · The reconciliation register as standing artifact 7
- 5. YANTRA Engineering Sub-Specification 7**
 - 5.1 Architecture decision 7
 - 5.2 Input schema (SPEC) 8
 - 5.3 Computation (SPEC) 9
 - 5.4 Output schema (SPEC) 10
 - 5.5 Confidence bands and the fallback ladder 10
 - 5.6 Determinism policy and versioning 10
 - 5.7 Test strategy 11
- 6. The Canonical Data Model 11**
 - 6.1 Enums (SCHEMA) 11
 - 6.2 Identity 12
 - 6.3 The Mirror 13
 - 6.4 The Spine 14
 - 6.5 The Ledger 15
 - 6.6 The Field 15
 - 6.7 Load-bearing relationships 16
 - 6.8 Seeding and consistency 16
- 7. Seeker State and Progress Semantics 17**
 - 7.1 Resolution rules 17
 - 7.2 Rendering contract 18
- 8. The Station Gate 18**

8.1 The hybrid gate (SPEC)	19
8.2 Evaluation (SPEC)	19
8.3 The transparency copy	19
8.4 Edge cases	19
9. Register Integrity	20
9.1 The mandatory framing line	20
9.2 Typographic differentiation	20
9.3 Tag lifecycle: the three-state model	21
9.4 Adjudication and bad-faith controls	21
9.5 The public contest surface	21
10. Error States and Failure Design	22
11. Content Operations	23
11.1 Editorial workflow	23
11.2 Cadence (solo-operator realistic)	23
11.3 The quotable-passage checklist (per content page)	23
11.4 DECISIONS.md — the standing reconciliation register	23
11.5 Content debt	24
12. Analytics Instrumentation	24
12.1 Event dictionary	24
12.2 Envelope and governance	25
13. Security: The Access-File Finding	26
13.1 Rotation runbook (P0 — today, before any code)	26
13.2 Standing policy	26
14. Deferred Concerns	27
14.1 Internationalization: IAST is the product	27
14.2 Offline: one surface, done properly	27
14.3 Accessibility: WCAG 2.2 AA as the floor	27
15. Phased Build Roadmap	28
16. Register of Open Decisions	29

1. Executive Summary

The V2.0 Master Build Specification is a remarkable document: it resolves its own terminology conflicts in a reconciliation register, grades its own evidence in four registers, and earns its architecture. The verdict of this companion is equally clear about what it does not do: it never crosses from **what** to **how**. YANTRA, the differentiating engine, is described entirely in prose. There is no data model anywhere in the document, despite a corpus of 56 folios, 20 patterns, 86 lexicon terms, 16 pantheon forces and 110 URLs that must relate to one another. The Station III to IV gate demands 21 consecutive logged sessions, a requirement that collides with the retention reality of any consumer product. And the deployment credentials that accompanied the specification package are live production secrets, exposed in plaintext, that must be rotated before another line of code ships.

This document is the engineering translation layer. It does not re-argue the architecture — the reviewer’s finding stands that the intellectual core (the register taxonomy, the three-gate system, the Dossier-as-entry rite, the reconciliation register) is genuinely defensible IP. Instead, each gap identified in review is converted into a buildable artifact: a complete sub-specification for YANTRA with input and output schemas, confidence bands and a fallback ladder; a canonical 19-table data model expressed in Prisma SDL and mapped to the deployed Turso stack; a SeekerState contract that resolves the linear-Spine-versus-circular-Doors tension; a retention-safe reformulation of the station gate; a three-state adjudication workflow for register tags; an error-state catalogue with dignified failure copy; a content operations pipeline for a solo archivist; a two-lattice analytics dictionary; and a five-phase roadmap with acceptance tests.

0 → 19

schema artifacts specified in §6 (none exist today)

5 layers

YANTRA engine architecture, deterministic core (§5)

21-in-30

rolling gate + PARĪKṢĀ preview replaces the cliff (§8)

Immediate actions, in order

- **Rotate every credential in the access file today** — GitHub PAT, Cloudinary, OpenRouter, Turso auth token, Vercel token, and the admin password set to a placeholder. All are compromised by transmission; see §13.
- **Ratify or amend the Register of Open Decisions (§16)** — six decisions (gate model, YANTRA provider posture, adjudicator, IP protection, door-scoped stations, guru export) gate the build sequence.
- **Create the constants file before any feature code** — the canonical counts (56/20/86/16/8/4/6/4) become a single imported module, per Dossier No. 03’s engine doctrine (§4, §11).
- **Adopt the data model in §6 as the contract** between YANTRA, the Chambers, the Spine and the Ledger — the reviewer’s recommendation, implemented.

How to read the result: sections 3 and 4 are the diagnosis (what is missing, what is strong and what each strength obligates). Sections 5 through 12 are the normative specs — the how. Sections 13 through 16 are operations: security posture, deferred concerns, sequencing, and the decisions only the owner can make. Every

normative statement uses RFC 2119 language so the developer can distinguish a requirement from a recommendation.

2. How to Read This Document

This companion sits between two existing documents and never contradicts either. The **V2.0 Master Build Specification** remains the authority on *what* the platform is: the Chambers, the Spine, the Doors, the registers, the covenant tiers. **Dossier No. 03 — The Visibility Engine** remains the authority on *why* the machine must also win citations and convert them honourably: the GEO kit, the karma cluster, the funnel doctrine, the kalki-geo-ops defense machine. This document supplies the third thing neither contains: *how* to build the product layer beneath both. Where the V2.0 spec and this companion appear to disagree, the disagreement is recorded in §16 as an open decision, in the same table format the spec’s own reconciliation register made canonical.

2.1 Severity model

Label	Meaning	Action it demands
CRITICAL	Blocks development. Three developers would build three incompatible things.	Resolve before Phase 1 code. Specs provided in §5–§6, §13.
TENSION	Conceptually sound, structurally unresolved. Surfaces as ambiguity mid-build.	Adopt the resolution contract in §7–§9.
STRENGTH	Defensible IP worth protecting and encoding.	Meet the engineering obligations in §4.
DEFERRED	Real but not load-bearing for v1.	Position taken in §14; revisit at the stated trigger.

Table 2.1 — Severity labels used throughout this document.

2.2 Citation and artifact conventions

- **spec §x.y** — an anchor into the V2.0 Master Build Specification, cited as reviewed (the reviewer’s § references are treated as authoritative).
- **D03 Part x** — Dossier No. 03, The Visibility Engine (v1.0, August 2026).
- **G-xx** — a gap from the register in §3; **D-xx** — an open decision from §16; **S-xx** — a strength from §4.
- **MUST / SHOULD / MAY** — RFC 2119 semantics. A MUST is a build-acceptance criterion; a SHOULD may be traded with a recorded decision; a MAY is optional.
- Artifact levels: **SPEC** (normative, implement as written), **SCHEMA** (canonical draft, adopt then evolve via migration), **TABLE** (reference data).

One convention borrowed deliberately: identifiers in code are ASCII (PARIKSHA, KALI), while display strings carry full IAST (PĀṚIKṢĀ, Kālī) from the constants file. The spec’s position that Sanskrit terms are first-class vocabulary is preserved where seekers read, and the compiler is kept happy where machines read. Every display string therefore lives exactly once, in the constants module, beside the canonical counts D03

Engine II made a single source of truth.

3. The Gap Register

Thirteen gaps, each traced to its source, each resolved in a named section of this document. Ten originate in the review's critical and structural findings and its missing-sections table; one (G-10) is new, discovered in the materials themselves; two restate review strengths as engineering obligations where the obligation outgrew §4. The register is the document's index: find the row, read the section, build the artifact.

ID	Severity	Area	The gap	Resolved in	Phase
G-0 1	CRITICAL	YANTRA engine	Differentiating engine described in prose; no computation, schema, fallback or determinism policy. (spec §5)	§5 SPEC	P2
G-0 2	CRITICAL	Data model	No schema for 56 folios, 20 patterns, 86 terms, 16 forces, dossiers, logs, tags, badges, tiers. (spec §7 absent)	§6 SCHEMA	P1
G-0 3	CRITICAL	Station gate	21 consecutive sessions = 5–7 weeks to Station IV; no grace, no alternate path, no preview. (spec §4)	§8 SPEC	P3
G-0 4	TENSION	Spine vs Doors	Linear Spine and ring Doors coexist with no SeekerState contract; progress-map semantics undefined. (spec §2)	§7 SPEC	P3
G-0 5	TENSION	PRATIBIMBA	Interpretive register typeset with the dignity of evidence; horoscope risk the Codex itself warns against.	§9 SPEC	P1
G-0 6	TENSION	Tag adjudication	Contestation promised (spec §4.2) with no workflow, SLA, or bad-faith control.	§9 SPEC	P1
G-0 7	TENSION	Content ops	No authorship, editorial states or cadence for case files and folio deepening.	§11 SPEC	P4
G-0 8	CRITICAL	Error states	No defined behaviour when YANTRA, Dossier generation, or the logger fail.	§10 SPEC	P2
G-0 9	TENSION	Analytics	No event dictionary; spine progression and conversion unmeasurable.	§12 SPEC	P4
G-1 0	CRITICAL	Secrets	Live production credentials circulated in plaintext as an unencrypted PDF (the access file).	§13 SPEC	P0
G-1 1	DEFERRED	i18n	IAST transliteration only; no Hindi or Indic language strategy stated.	§14.1	post
G-1 2	DEFERRED	Offline/PWA	Sādhana happens in ashrams; logger is the one surface that must work offline.	§14.2	P3
G-1 3	DEFERRED	Accessibility	No WCAG target, screen-reader or reduced-motion specification.	§14.3	P1 floor

Table 3.1 — The gap register. Phases reference the roadmap in §15; P0 precedes all code.

Read as a set: three criticals are absence-of-artifact problems (an engine, a schema, an error model) and are fully discharged by §5, §6 and §10; one critical (G-10) is an operational emergency independent of the build; the tensions are all resolvable by contract — a state object, a formula, a lifecycle — rather than by redesign;

and the deferred items are genuine v1 exclusions with named revisit triggers, not oversights. Nothing in the register requires re-litigating the spec’s architecture. That is the strongest statement this document can make about it.

4. Strengths and Their Engineering Obligations

A strength that is not encoded as a constant, a schema or a workflow will erode during the build — not through disagreement but through a hundred small conveniences. Each strength from the review is therefore restated as obligations with acceptance criteria, so the IP survives contact with the compiler.

4.1 S-01 · The four-register taxonomy

ĀGAMA / ANUBHAVA / PARĪKṢĀ / PRATIBIMBA is the strongest asset in the corpus because it is simultaneously classical and practical: it maps almost one-to-one onto pramāṇa theory, which no competitor can say of a content farm’s category list, and it is filterable, taggable and visually distinct in a way testimony-led genre sites never attempt. The mapping also gives the Ledger its public explanation: the registers are not editorial mood, they are epistemology.

Pramāṇa (source of knowledge)	Register	What it certifies
Śbda — reliable testimony	ĀGAMA	Textual authority: what the tradition and its sources actually say.
Anubhava — lived experience	ANUBHAVA	Practitioner testimony: what a witness reports, unverifiable but honest.
Pratyakṣa — direct perception	PARĪKṢĀ	Checkable evidence: what the reader can verify for themselves.
Pratibimba — reflection	PRATIBIMBA	Interpretation: a reading offered as a lens, never as fact (guarded in §9).

Table 4.1 — The register taxonomy mapped to its classical roots.

- Obligation A: the four registers exist as a closed enum in the schema (§6.3) and as constants with IAST display names; no content ships without exactly one register value. **MUST**.
- Obligation B: register is a filter dimension everywhere content is listed — the Ledger, the Atlas, the karma cluster — instrumented via `register_filter_used` (§12). **MUST**.
- Obligation C: the taxonomy enters the trademark docket this quarter (decision D-04, §16) — classification methodology is the defensible claim; patents are a slower, weaker instrument for it. **SHOULD**.

4.2 S-02 · The Dossier as rite of entry

Making the Dossier the first act — before any content, before any account pressure — is the review’s strongest UX finding, and D03 Part VII confirms it commercially: the Dossier is the platform’s most distinctive instrument and its natural self-recognition moment. Its engineering obligations are persistence (the

review’s risk: a Dossier that cannot persist has no table), reproducibility (every Dossier is a recorded YantraRun), versioning (re-sittings create versions, they do not overwrite history), and export. The **share-with-your-guru** export the review proposes becomes a first-class artifact: a typeset PDF carrying the seeker’s patterns, the evidence-grade separation of what is known versus read, and the mirror-framing line — a document designed to make the teacher, not the platform, the next conversation.

4.3 S-03 · The reconciliation register as standing artifact

The spec’s conflict-resolution table — Śravaṇa chosen over Vidyā to avoid collision, Sāmarasya over Samāpatti because “corporate words cannot carry sacred weight” — is best-in-class project management precisely because it converts future arguments into one-line records. The obligation is to keep it alive: the register becomes `DECISIONS.md` in the repository, seeded with the spec’s own decisions, extended by every decision this document leaves open (§16), and governed by the entry format in §11.5. A decision not in the file did not happen.

5. YANTRA Engineering Sub-Specification

The spec’s claim that “no competitor has an engine of this shape” is only an asset if the engine’s shape is written down. YANTRA is specified here as a five-layer pipeline with one non-negotiable property: **classification is deterministic; only prose is generated**. The seeker’s patterns are computed by an inspectable, versioned, test-covered matrix — never assigned by a language model. The model writes paragraphs about scores it cannot change. This division is what makes the engine defensible IP: its judgements can be re-run, audited and explained, which an LLM-in-the-loop classifier can never promise a seeker whose mirror reading might otherwise shift between two sittings.

5.1 Architecture decision

Candidate	Deterministic	Explainable	Cost/latency	Verdict
Rule-based classifier (if X then pattern Y)	Yes	Trivially	Lowest	Rejected — brittle; transit geometry collapses into if-chains.
LLM-only (prompt in, patterns out)	No	Post-hoc only	Per-call	Rejected — non-reproducible; the horoscope problem at machine scale.
Hybrid: scoring matrix + LLM prose	Yes (core)	Scores + weights	Low core, one LLM call	Adopted — inspectable classification, generated narrative.

Table 5.1 — Why the matrix decides and the model writes.

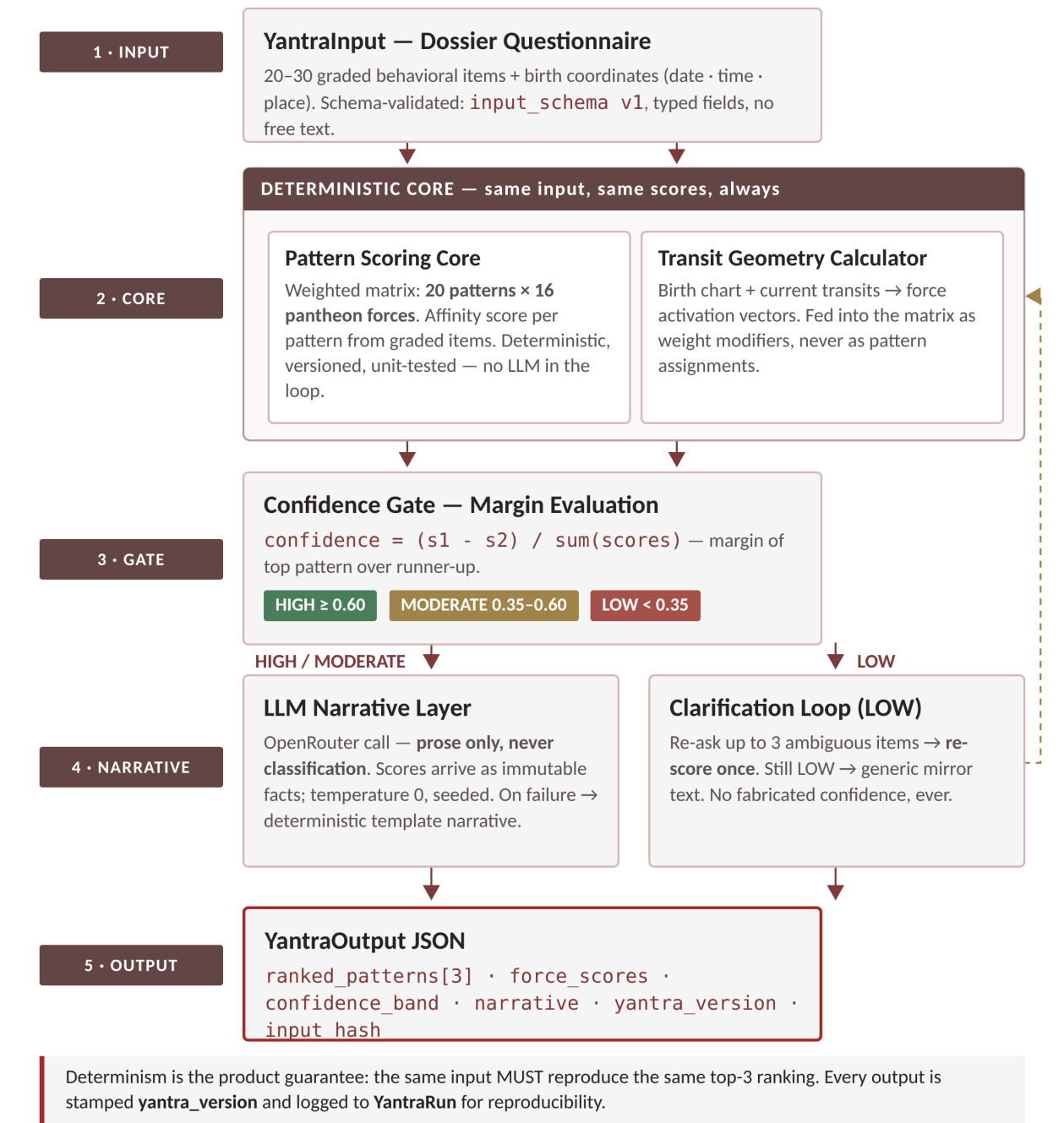


Figure 5.1 — YANTRA dataflow. The deterministic core computes; the LLM narrates; the confidence gate decides who is allowed to do either.

5.2 Input schema (SPEC)

One questionnaire sitting produces one immutable **YantraInput**. Items are graded sliders normalized to [0, 1] on named axes — no free text reaches the engine, which is what keeps the core deterministic and the audit hash meaningful. Birth coordinates feed only the transit calculator.

YANTRA INPUT — SCHEMA V1 (JSON)

```
{
  "$schema": "yantra/input/v1",
  "seekerRef": "usr_cuid",
  "items": [
    { "id": "q01", "axis": "self_structure", "grade": 0.0 },
    { "id": "q02", "axis": "self_structure", "grade": 0.25 },
    { "id": "q03", "axis": "relational_loop", "grade": 0.75 }
  ],
  "birth": { "date": "1990-04-17", "time": "03:42",
    "place": "varanasi_in", "tz": "Asia/Kolkata" },
  "askedAt": "2026-08-29T04:12:00Z"
}
```

5.3 Computation (SPEC)

The core is arithmetic: a seeded weight matrix over 20 patterns × 16 pantheon forces, modulated by transit vectors, summed over graded items, squashed to affinities. Confidence is the *margin* of the top pattern over the runner-up — a measure of how decided the engine is, not how enthusiastic. The pseudocode below is normative for v1:

YANTRA DETERMINISTIC CORE (PSEUDOCODE)

```
# pseudocode – deterministic core (temperature: none)
def score(input: YantraInput) -> Scores:
  w = WEIGHTS          # matrix[20 patterns x 16 forces], seeded, versioned
  v = transit_vectors(input.birth, input.askedAt)
  # transit modifies weights; it NEVER assigns patterns directly
  wmod = w * (1 + VTAU * v)      # VTAU = 0.15, clamps [-0.15, +0.15]
  a = {}                  # pattern affinities
  for p in PATTERNS:      # 20 rows, seeded from constants
    s = sum(input.grade(i) * wmod[p, axis_of(i)] for i in items_of(p))
    a[p] = sigmoid(s / norm(p))  # affinity in [0, 1)
  top, second = two_highest(a)
  margin = (top - second) / (top + second + EPS)
  return Scores(a, v, top, second, margin)

def narrate(scores) -> Narrative:
  if scores.margin >= 0.60: band = HIGH
  elif scores.margin >= 0.35: band = MODERATE
  else: band = LOW
  if band == LOW:
    reask = clarify(3)      # <= 3 ambiguous items, then ONE re-score
    if reask.margin >= 0.35: band, scores = reask.band, reask
    else: return template(GENERIC_MIRROR, no fabricated confidence)
  try:
    text = llm_prose(scores, seed=fixed, temperature=0) # prose ONLY
  except LLMUnavailable:
    text = template_narrative(scores) # deterministic fallback
  return Narrative(band, text, source)
```

5.4 Output schema (SPEC)

Every run writes one `YantraOutput` — ranked top-3 patterns with affinities and force attributions, the full force score map, the confidence band, the narrative and its source, and the audit stamp. The `inputHash` plus `yantraVersion` pair makes any Dossier reproducible years later:

YANTRA OUTPUT – SCHEMA V1 (JSON)

```
{
  "$schema": "yantra/output/v1",
  "yantraVersion": "1.0.0",
  "inputHash": "sha256:...",
  "confidence": { "band": "HIGH", "margin": 0.64 },
  "rankedPatterns": [
    { "slug": "hungry-ghost", "affinity": 0.81, "forces": ["mangala", "rahu"] },
    { "slug": "strong-child", "affinity": 0.17, "forces": ["surya"] },
    { "slug": "orphan-gift", "affinity": 0.02, "forces": ["chandra"] }
  ],
  "forceScores": { "mangala": 0.72, "rahu": 0.66, "chandra": 0.31 },
  "narrative": { "source": "LLM",
                 "text": "... " },
  "audit": { "runId": "run_cuid", "durationMs": 1820 }
}
```

5.5 Confidence bands and the fallback ladder

Band	Margin	Engine behaviour	Seeker sees
HIGH	≥ 0.60	Narrate via LLM with scores as immutable facts.	Full Dossier reading.
MODERATE	0.35–0.60	Narrate via LLM; flag reduced specificity in copy.	Full reading, honest hedge.
LOW	< 0.35	Clarification loop: re-ask ≤ 3 ambiguous items, re-score once.	Up to 3 sharper questions.
LOW (persisting)	< 0.35 after re-score	No fabricated confidence. Generic mirror text, template source.	A true “the mirror needs a clearer moment” page.
LLM failure	any band	Template narrative composed deterministically from scores.	Identical reading, zero dependency on the provider.

Table 5.2 — Confidence bands and fallback ladder (G-01, G-08).

5.6 Determinism policy and versioning

- Same `inputHash` + same `yantraVersion` MUST reproduce the identical top-3 ranking and affinities. The LLM call runs at temperature 0 with a fixed seed; narrative is cached per (`inputHash`, `version`).
- The weight matrix lives in version control as data; every change bumps `yantraVersion` and requires the golden set to pass.

- YantraRun rows are append-only. A re-sitting creates a new Dossier version and a new run; history is never rewritten (S-02 obligation).

5.7 Test strategy

- **Golden set:** 30 fixture inputs with frozen expected outputs (top-3, band, force attributions). CI fails on any drift — this is the determinism contract made mechanical, in the same spirit D03 made the consistency firewall mechanical.
- **Adversarial fixtures:** all-neutral grades (must yield LOW), contradictory grades (must yield LOW + clarification), duplicated submissions (must dedupe to one run), provider outage (must fall to template narrative in < 2s).
- **Narrative guardrails test:** LLM output is asserted to contain the ranked pattern names and no register-violating claims (no “you are” pathology statements — the framing rules of §9 apply to generated prose too).

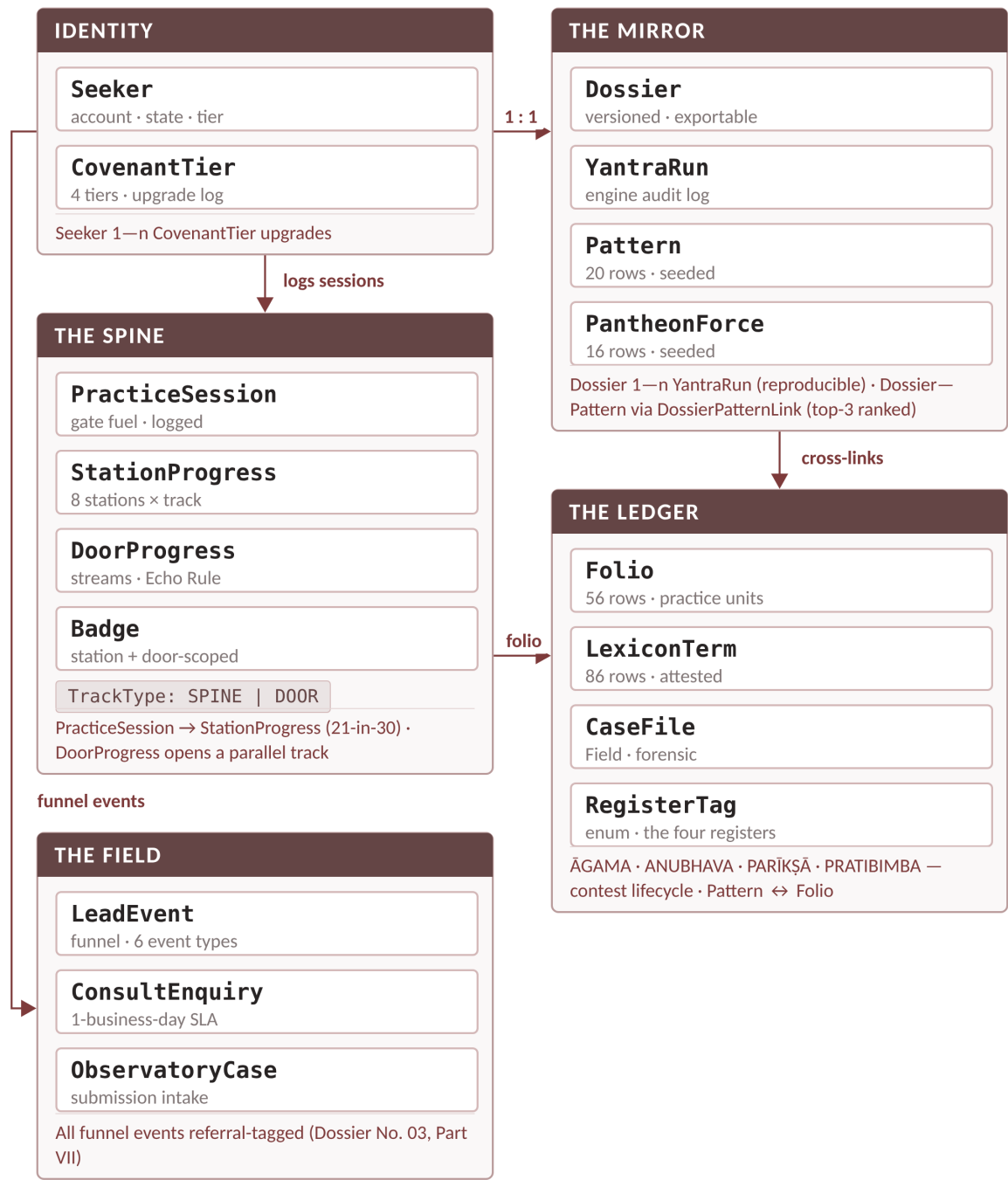
6. The Canonical Data Model

The deployed stack is Next.js on Vercel with a Turso (libsql) database — facts established by the access file and D03’s deployment record. The model is therefore authored for **Drizzle ORM with @libsql/client**, but presented in Prisma SDL because SDL reads as a contract: it is the artifact three developers can read identically. The mapping is mechanical (Prisma model → Drizzle pgTable-style definition; enums → check-constrained text columns or TS const arrays). What **MUST** survive the translation is the relationship structure: the logger feeds the gate; the tag carries its contest state; the Dossier points at an immutable run.

6.1 Enums (SCHEMA)

ENUMS — PRISMA SDL (ASCII IDENTIFIERS)

```
// enums.zmodel — identifiers ASCII; display names via constants
enum Register      { AGAMA ANUBHAVA PARIKSHA PRATIBIMBA }
enum TagState      { ASSIGNED CONTESTED CONFIRMED REVISED }
enum TrackType     { SPINE DOOR }
enum DoorStream    { KALI AGHORA SHAKTA }
enum CovenantTier  { SEEKER PRACTITIONER COVENANT CIRCLE }
enum BandState     { HIGH MODERATE LOW }
enum NarrativeSrc  { LLM TEMPLATE GENERIC }
enum EventName     {
  MANUAL_DOWNLOADED DOSSIER_COMPLETED OBSERVATORY_SUBMITTED
  CONSULT_ENQUIRED  COURSE_ENROLLED   TIER_UPGRADED
  GATE_EVALUATED    GATE_CROSSED      GATE_STALLED
  STATION_REACHED   DOOR_OPENED       REGISTER_FILTER_USED
  TAG_CONTESTED     PRATIBIMBA_VIEWED YANTRA_FALLBACK_SHOWN
}
```



Simplified domain map — 15 entities, 4 enums. The full field-level schema (Prisma SDL, Drizzle-mapped) is specified in §6 of this document.

Figure 6.1 — Domain map. Fifteen entities, four enums, five domains; arrows mark the load-bearing inter-domain relations.

6.2 Identity

IDENTITY — PRISMA SDL

```
// IDENTITY -----
model Seeker {
```

```

    id          String    @id @default(cuid())
    email       String?   @unique
    displayName  String?
    tier         CovenantTier @default(SEEKER)
    createdAt   DateTime  @default(now())
    dossier     Dossier?
    upgrades    CovenantUpgrade[]
    sessions    PracticeSession[]
    stations    StationProgress[]
    doors       DoorProgress[]
    badges      Badge[]
    leads       LeadEvent[]
    enquiries   ConsultEnquiry[]
  }
  model CovenantUpgrade {
    id          String    @id @default(cuid())
    seekerId    String
    fromTier    CovenantTier
    toTier      CovenantTier
    at          DateTime  @default(now())
  }

```

6.3 The Mirror

THE MIRROR – PRISMA SDL

```

// THE MIRROR -----
model Dossier {
  id          String    @id @default(cuid())
  seekerId    String    @unique
  version     Int       @default(1)    // re-sit appends, never overwrites
  runId       String    @unique
  run         YantraRun @relation(fields: [runId])
  createdAt   DateTime  @default(now())
  sections    DossierSection[]
  patterns    DossierPatternLink[]
}
model YantraRun {
  id          String    @id @default(cuid())
  seekerId    String
  inputJson    Json
  outputJson   Json
  yantraVersion String
  band         BandState
  narrativeSrc NarrativeSrc
  inputHash    String  // sha256(canonical(inputJson))
  createdAt    DateTime @default(now())
}
model DossierPatternLink {
  dossierId String
  patternId String
  rank      Int      // 1..3
  affinity  Float
}

```

```

    @@unique([dossierId, patternId])
  }
  model Pattern {          // 20 rows, seeded from constants
    id      String @id
    slug    String @unique
    folios  FolioPattern[]
  }
  model PantheonForce {    // 16 rows, seeded from constants
    id      String @id
    slug    String @unique
  }

```

6.4 The Spine

THE SPINE – PRISMA SDL

```

// THE SPINE -----
model PracticeSession {          // THE gate fuel – logger certifies
  id      String  @id @default(cuid())
  seekerId String
  folioId String?           // what was practiced
  door    DoorStream?       // if inside a door
  loggedAt DateTime @default(now())
  localDate String          // YYYY-MM-DD, seeker-local (see 8.6)
  minutes  Int?
  @@index([seekerId, localDate])
}
model StationProgress {          // 8 stations x track
  id      String  @id @default(cuid())
  seekerId String
  station Int        // 1..8
  track    TrackType
  door     DoorStream? // required when track = D00R
  status   String    // LOCKED | ACTIVE | EARNED
  earnedAt DateTime?
  @@unique([seekerId, station, track, door])
}
model DoorProgress {             // Echo Rule: opens at Pravesa,
  id      String  @id @default(cuid()) // closes at Samarasya
  seekerId String
  door    DoorStream
  openedAt DateTime @default(now())
  closedAt DateTime?
  @@unique([seekerId, door])
}
model Badge {
  id      String  @id @default(cuid())
  seekerId String
  station Int?
  track    TrackType?
  door     DoorStream?
  earnedAt DateTime @default(now())
}

```

}

6.5 The Ledger

THE LEDGER – PRISMA SDL

```
// THE LEDGER -----
model Folio {          // 56 rows
  id          String @id
  slug        String @unique
  title       String
  body        String
  registers   FolioTag[]
  patterns    FolioPattern[]
}
model FolioTag {       // the contested surface (see 9.3)
  id          String   @id @default(cuid())
  folioId     String
  register     Register
  state       TagState @default(ASSIGNED)
  contestedBy String?   // seeker id if contested
  reason      String?
  contestedAt DateTime?
  adjudicatedAt DateTime?
  note        String?   // archivist rationale
  @@unique([folioId, register])
}
model FolioPattern { @@unique([folioId, patternId]) }
model LexiconTerm {   // 86 rows, DefinedTermSet source (D03 L2)
  id          String @id
  slug        String @unique
  termIAST    String   // display, e.g. "pra_rabdha"
  gloss       String
  attestation String   // textual source, AGAMA-grade
}
model CaseFile {      // Field / Forensics, per spec Part V
  id          String @id
  slug        String @unique
  title       String
  body        String
  register     Register
  verdict      String // 4 verdict states
  published    DateTime?
}
```

6.6 The Field

THE FIELD – PRISMA SDL

```
// THE FIELD -----
model ObservatoryCase {
```

```

    id      String   @id @default(cuid())
    seekerId String?
    subject String
    body     String
    state    String   // SUBMITTED | UNDER_REVIEW | PUBLISHED
    at       DateTime @default(now())
  }
  model ConsultEnquiry {           // 1-business-day SLA (D03 VII)
    id      String   @id @default(cuid())
    seekerId String?
    email    String
    question String
    sourcePage String
    createdAt DateTime @default(now())
    respondedAt DateTime?
  }
  model LeadEvent {                // the analytics lattice (12)
    id      String   @id @default(cuid())
    seekerId String?
    event    EventName
    path     String
    referrer String? // ai-referral attribution (D03)
    props    Json?
    at       DateTime @default(now())
    @@index([event, at])
  }
}

```

6.7 Load-bearing relationships

Relation	Why it is load-bearing	Failure if missing
PracticeSession → StationProgress	The logger certifies the gate — spec’s own principle. Without it, gates run on claims.	Gate evaluation has no source of truth; retention analytics lie.
Dossier → YantraRun (1:1, immutable)	Reproducibility of every mirror reading ever issued.	Dossiers become unauditable; regeneration silently rewrites history.
FolioTag(register, state)	Register filtering + contest lifecycle on every content card.	Folios cannot carry tags; the Ledger’s public methodology page has nothing to point at.
StationProgress(track, door?)	Universal spine and door tracks coexist (Echo Rule, §7).	Door progress either erodes the spine or is invisible; badges render wrong.
LeadEvent(referrer)	D03’s attribution: AI-referred vs karma-page sessions must be distinguishable.	The two lattices cannot explain each other; the funnel is unmeasurable.

Table 6.1 — The five relationships that ARE the product contract.

6.8 Seeding and consistency

Pattern (20), PantheonForce (16), LexiconTerm (86) and Folio (56) are seeded from the constants file, never hand-typed — D03 Engine II’s doctrine applied to the database. A migration assertion **MUST** fail if seeded counts deviate from canonical, and the weekly geo-monitor run already checks the public counts (56/20)

against the same source. Internal inconsistency is a defect in the product; with the schema in place, it is also a defect the machines catch first.

7. Seeker State and Progress Semantics

The spec holds two true things in tension: the Spine is explicitly linear ($I \rightarrow VIII$) and the Doors are explicitly a ring — “a ring, never a ladder” — with the Echo Rule that every door re-opens Praveśa and closes at Sāmarasya. The tension is not a contradiction to resolve but a state machine to define. The answer is that the spine is the base axis of progress and a door is an **overlay track**: entering a door opens a second StationProgress sequence that runs parallel to the universal one, starts at Praveśa, and earns its own stations up to Sāmarasya. Both truths are preserved because both are tracked.

SEEKER-STATE CONTRACT (TYPESCRIPT)

```
// seeker-state.ts – the one resolver the UI is allowed to query
export interface SeekerState {
  station: 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8; // universal spine position
  stream: DoorStream | null; // active door, else null
  doorEpoch: number | null; // which Echo-cycle of the door
  stationsByTrack: Array<{ // parallel tracks (Echo Rule)
    track: 'SPINE' | 'DOOR';
    door: DoorStream | null;
    station: number;
    status: 'LOCKED' | 'ACTIVE' | 'EARNED';
  }>;
  sessionsWindow: { // gate fuel, see 8.3
    inWindow: number; // sessions in rolling 30d
    required: number; // 21
    graceTokens: number; // available, max 2/quarter
  };
  tier: CovenantTier;
  badges: Array<{ station?: number; door?: DoorStream; kind: BadgeKind }>;
  previewUnlocked: boolean; // PARIKSHA preview of Station IV+
}
```

7.1 Resolution rules

- **Universal spine**: every seeker has StationProgress rows with `track = SPINE` for stations 1–8. These are the public, badge-carrying stations. **MUST**.
- **Door overlay**: entering a door writes a DoorProgress row and a fresh set of `track = DOOR` rows beginning at Praveśa — the Echo Rule, encoded. Completion at Sāmarasya closes the DoorProgress row. **MUST**.
- **Badges**: station badges are universal; door-scoped badges render as “Station III · Kālī” — the review’s dual-badge question answered by composite display, not by duplicate badge entities. **MUST**.
- **One resolver**: the UI never assembles seeker state ad hoc; it calls `getSeekerState()`, which returns the contract above. Every screen — progress map, logger, Dossier header — renders from the same

object. MUST.

7.2 Rendering contract

UI surface	Spine (track = SPINE)	Active door (track = DOOR)
Progress map	Rendered dim, always visible — the universal skeleton.	Rendered bright, overlaid on the same map; the ring reads as an orbit around the spine.
Station card	Content universal; framing neutral.	Content universal; framing door-tinted (a Kālī seeker sees the same Station III folio with door-voiced framing).
Badges	“Station III” (universal).	“Station III · Kālī” (composite, never a separate entity).
Mixed progress	Spine V in Aghora shown beside Spine II in Śākta — both tracks render simultaneously, stacked by door.	No hiding, no averaging; the map shows every earned track at once.

Table 7.1 — How one state object renders everywhere (G-04).

The three questions from review resolve cleanly under this contract. A seeker in the Kālī door sees the *same* Station III content as an Aghora stream seeker — the corpus is universal — but framed in the door’s voice. Universal and door-scoped badges coexist as composite labels. And a seeker at Station V in Aghora but Station II in Śākta simply has two bright tracks on one map: the ring and the ladder were never the same object, and now the schema says so.

8. The Station Gate

The principle is not negotiable and this document does not negotiate it: **stations are earned, and the logger — not a claim — certifies the gate**. What fails is the arithmetic around the principle. Twenty-one consecutive sessions at a realistic 3–4 sittings per week is five to seven weeks before a seeker sees Station IV, while consumer retention curves mostly die by week three. A gate that certifies devotion by extinguishing the devoted has confused consecutiveness with commitment. The fix keeps the earned-ness and replaces the cliff.

Model	Time to Station IV (3–4/wk)	Missed day	Verdict
Strict 21 consecutive (spec)	5–7 weeks	Reset to zero — week-20 lapse costs everything	Retention cliff; principled, punitive.
Rolling 21-in-30	3–4 weeks	Absorbed by the window	Honest; loses the “candle stayed lit” narrative.
Hybrid (adopted)	3–4 weeks	Grace tokens, then window absorbs	Earned-ness intact; cliff removed.

Table 8.1 — Gate models compared. The hybrid is recommended (D-01).

8.1 The hybrid gate (SPEC)

- **Rolling window:** the III→IV gate opens at 21 logged sessions within any 30-day window (seeker-local dates). MUST.
- **Grace tokens:** 2 tokens per quarter, each converting one missed day into a logged one. Tokens are visible, earned by presence, never purchasable — the commercial copy never mixes the two (D03 VII). SHOULD.
- **PARĪKṢĀ preview:** while the gate is closed, Station IV reading content is visible in **preview register** — evidence-grade (PARĪKṢĀ) material only. The gate gates *practice*; reading was never meant to be rationed. MUST.
- **Transparency copy:** the gate UI states why the gate exists and current progress (“14 of 21 in your window”). The principle is visible, so it reads as doctrine rather than dysfunction. MUST.

8.2 Evaluation (SPEC)

GATE EVALUATION (PSEUDOCODE)

```
def evaluate_gate(seeker, at = today_local(seeker)):
    days = distinct_local_dates(seeker.sessions, window=30d, end=at)
    # tokens can mark <= 2 missed days per quarter as present
    days += redeemable_tokens(seeker, quarter(at), cap=2)
    crossed = len(days) >= 21
    emit(GATE_EVALUATED, {in_window: len(days), crossed})
    if crossed and station_row(seeker, 4).status != EARNED:
        earn(seeker, 4, track=SPINE); emit(GATE_CROSSED, {station: 4})
    if stalled(days, weeks=2): emit(GATE_STALLED, {hint: preview_register})
    return { crossed, inWindow: len(days), required: 21 }
```

8.3 The transparency copy

“The Station IV threshold is twenty-one sittings inside thirty days — not because the count is sacred, but because practice that returns is what changes a pattern. Your window holds 14 of 21. Miss a day and nothing is lost; the window moves with you. The reading is already open to you — the practice is what is earned.”

In-product gate copy, draft for the developer to tune — the seeker-facing answer to “why does this wall exist?”

8.4 Edge cases

Case	Rule
Timezone	Sessions are dated seeker-local (localDate). A sitting at 23:50 IST and the next at 00:10 IST are two days, not one.
Multiple sittings per day	Distinct local dates count once each; the gate measures return, not volume.

Case	Rule
Backfill	Manual backdated entries are allowed within 72h and flagged in the log; older backfill is rejected.
Window boundary	A session counts for 30 days from its local date; evaluation is deterministic at read time (no cron drift).
Door tracks	Door gates use the same formula on door-scoped sessions; universal spine progress is never reset by door entry.

Table 8.2 — Gate edge cases (G-03).

9. Register Integrity

The register taxonomy’s honesty depends on its weakest register being handled with more care, not less, than its strongest. PRATIBIMBA is where the platform tells a seeker “this is the loop it points to in you” — which is, structurally, exactly what a horoscope does. The Codex itself contains a chapter on the danger (“The Illusion of the Horoscope”): personality typing becomes a cage the moment the map is mistaken for the territory. The guardrails below are therefore not legal hygiene; they are the platform taking its own intellectual position seriously enough to encode it in the product.

9.1 The mandatory framing line

“A mirror reading is a lens, not a diagnosis. It reflects one pattern among many. The tradition’s warning: the map is not the territory.”

The PRATIBIMBA framing line — MUST appear on every mirror-reading surface.

- Where it MUST appear: every Dossier PRATIBIMBA section, every Pattern Atlas mirror essay, every karma-cluster page carrying interpretive material (D03 Part II’s caution makes the karma cluster the first large-scale test), and the /vault/ledger methodology page.
- Generated prose obeys the same rule: YANTRA’s narrative guardrails (§5.7) forbid “you are” pathology statements; the mirror speaks in “this pattern points to”, never in identity verdicts.

9.2 Typographic differentiation

Register	Chip treatment	Rationale
ĀGAMA	Solid chip, serif small-caps	Authority: typographically heaviest claim.
ANUBHAVA	Solid chip, lighter weight	Testimony: honest, secondary.
PARĪKṢĀ	Solid chip, accent border	Evidence: checkable, visually crisp.
PRATIBIMBA	Dashed border, italic	Reading: the chip itself signals provisionality before a word is read.

Table 9.1 — Same dignity, different epistemic weight: the strongest IP underlined by its own typography (G-05).

9.3 Tag lifecycle: the three-state model

The spec promises that register tags can be contested but defines no workflow (“how to contest one”, spec §4.2). The minimal honest machinery is a three-state lifecycle on the FolioTag table — assigned, contested, resolved — with the state visible to readers. Contesting a tag does not silently change it; it marks it as under review, in public, exactly as the Ledger’s own methodology page should show its work.

State	Who acts	What happens	Reader sees
ASSIGNED	Archivist at publish	Tag set with editorial rationale in the log.	Normal chip.
CONTESTED	Any seeker, via form	Contest recorded with reason + citation; adjudication clock starts.	Chip with “under review” badge.
CONFIRMED	Archivist (SLA 14 days)	Original upheld; note appended.	Chip + “reviewed” tick.
REVISED	Archivist (SLA 14 days)	Tag changed; errata entry published in the Ledger.	New chip + errata line.

Table 9.2 — Tag lifecycle (G-06). Every transition is logged.

9.4 Adjudication and bad-faith controls

- **Adjudicator:** the archivist (solo at v1; committee option remains open, D-03). Adjudication is a named role in the schema, so it can be delegated later without re-engineering. **MUST.**
- **SLA:** 14 days from CONTESTED to resolution, visible on the badge. A methodology page that promises review and delivers silence is worse than one that promises nothing. **MUST.**
- **Evidence requirement:** a contest **MUST** cite its grounds — textual citation for ĀGAMA disputes, verifiable source for PARĪKṢĀ disputes, or argued reading for PRATIBIMBA disputes. Forms without citations are rejected at submission. **MUST.**
- **Upgrade requests:** ANUBHAVA → PARĪKṢĀ or → ĀGAMA upgrades are auto-rejected unless accompanied by a checkable citation. Testimony claiming to be scripture is the exact confusion the registers exist to prevent. **MUST.**
- **Rate limits:** max 3 open contests per seeker; serial re-contesting of a resolved tag is throttled. **SHOULD.**

9.5 The public contest surface

The methodology page at `/vault/ledger` carries the contest form itself: content URL, current register, proposed register, citation URL, free-text rationale (500 chars), and a statement of the review criteria — the same table 9.2 lifecycle and 9.4 rules, published. The form’s existence is itself an argument: a platform that grades evidence and invites its gradings to be contested is making the strongest available claim about its own epistemic hygiene.

10. Error States and Failure Design

The spec defines the product’s sacred register but never its failure register. The design principle here is drawn from the platform’s own voice: an error message is a moment of the product speaking, so it must not break the container. No stack traces, no “something went wrong”, no blank screens. The seeker is told true things: what was saved, what is unavailable, what to do next. Where the domain allows, failure copy borrows the tradition’s own vocabulary of patience rather than the startup’s vocabulary of apology.

Component · failure	Detection	Seeker-facing experience	Recovery · integrity
YANTRA · margin LOW	Confidence gate (§5.5)	Clarification loop; if persisting, the honest generic mirror page.	No fabricated band, ever; run logged with band LOW.
YANTRA · provider down	Call timeout 8s / non-200	Identical Dossier via template narrative (source: TEMPLATE).	Scores unaffected — the core is local arithmetic.
Dossier · generation fails	YantraRun write error	“The mirror needs a clearer moment. Your answers are saved.”	Answers persisted; one-click retry; no orphan Dossier rows.
Practice log · write fails	Local queue ack missing	Entry stays in the offline queue with a visible “pending” state.	Idempotent upsert by (seeker, localDate, folio); no double-count.
Practice log · corrupted	Schema validation on read	The day renders as “unverified”; nothing silently counted.	Quarantined row flagged for review; gate ignores it.
Database unreachable	Health probe / retry budget	Static surfaces stay readable; logging and Dossier pause with honest notice.	Reads degrade to cached pages; writes queue client-side (§14.2).
Offline seeker (ashram)	navigator.onLine = false	Logger fully functional; everything else deferred.	IndexedDB queue syncs on reconnect; conflicts resolved by localDate.
Guru export fails	Render pipeline error	“The letter could not be prepared just now” + retry.	Dossier itself never mutated by export; failures isolated.

Table 10.1 — Failure catalogue (G-08). Detection is largely mechanical: D03’s geo-monitor already models this doctrine.

Two of these rows deserve emphasis. The **template narrative** is the reason the LLM sits at the edge of the pipeline and not inside it: the Dossier, the platform’s rite of entry, must never have a single external dependency that can take it down. And the **corrupted log row** encodes the gate’s philosophy at the storage layer: the logger certifies the gate, so the logger’s own failures must fail toward under-counting, never toward unearned progress.

11. Content Operations

The corpus ships with 56 folios at a median of 165 words, a 15-chapter karma manuscript awaiting its cluster (D03 Part IV), case files promised by the Field, and one operator. Content operations are specified here for that reality: a solo archivist with a workflow, not a newsroom. The design goal is that every editorial state a piece of content can be in is a column value, not a conversation.

11.1 Editorial workflow

State	Owner	Gate to next state
DRAFT	Author (archivist)	Complete draft with quotable passages present (§11.3).
REGISTER-TAGGED	Archivist	Exactly one register chosen with rationale; PRATIBIMBA pieces carry the framing line.
ATTESTATION REVIEW	Archivist (second read)	ĀGAMA claims carry citations; PARĪKṢĀ claims carry checkable sources.
PUBLISHED	(none)	Counts reconcile with the constants file; llms.txt section written the same day (D03 day-one rule).

Table 11.1 — Four editorial states; the register tag IS the review gate (G-07).

11.2 Cadence (solo-operator realistic)

- **Weekly:** one folio deepened to 300–500 words, working D03’s top-20-by-intent list in order. Depth where it converts; never a volume race.
- **Monthly:** one case file for the Field; one analytics half-hour reading the two lattices beside the citation tracker (D03 VI’s calendar, absorbed).
- **Quarterly:** refresh cycle on the weakest cluster; DECISIONS.md review; token/secret audit (§13).

11.3 The quotable-passage checklist (per content page)

- One or two self-contained definitions/passages of 40–60 words an engine could lift whole (D03 Engine III doctrine).
- H2s phrased as the real query; the answer in the first paragraph beneath.
- Counts and names consistent with the constants file — machine-checked, not remembered.
- Exactly one next step; no popups, countdowns, or scarcity mechanics anywhere (D03 Part VII doctrine).

11.4 DECISIONS.md — the standing reconciliation register

Field	Content
ID / date	D-07 · 2026-09-02
Decision	One line, imperative voice.
Options considered	The candidates that lost, one line each.
Rationale	The reason the winner won — spec §1 quality: “corporate words cannot carry sacred weight.”
Consequence	What this decision forecloses.
Status	OPEN · ADOPTED · SUPERSEDED (by D-xx).

Table 11.2 — DECISIONS.md entry format. Seeded with the spec’s own register (Śravaṇa over Vidyā; Sāmarasya over Samāpatti; 109+1 reconciliation).

11.5 Content debt

The honest ledger: 56 folios, of which the top 20 by search intent deepen over the first 20 editorial weeks; 16 karma-cluster pages launch as a territory claim (D03 Phase 1); case files begin at one per month from Phase 4. The karma manuscript itself ships as the lead magnet — the full PDF for one email — which makes the widest door in the genre also the list-builder (D03 4.2). Nothing in this schedule requires hiring before the platform proves the funnel; everything in it requires the workflow states above to exist first.

12. Analytics Instrumentation

The measurement stance follows the product’s doctrine: subtle does not mean unmeasured, it means honestly measured (D03 7.2). First-party event log only; no third-party pixels, no session recording, no dark-pattern telemetry. Every event carries its lattice (business or product), its referral path, and a privacy class. The two lattices explain each other — citations that do not convert point at thin landing pages; enquiries without citations point at audiences the machines have not found yet.

12.1 Event dictionary

Event	Trigger	Key properties	Lattice
manual_downloaded	Karma PDF delivered	page, referrer	business
dossier_completed	Dossier rendered	band, yantra_version	business
observatory_submitted	Case submitted	state	business
consult_enquired	Enquiry sent	sourcePage	business
course_enrolled	Enrollment	tier	business
tier_upgraded	Covenant change	from, to	business
gate_evaluated	Daily gate check	inWindow, required	product
gate_crossed	Station earned	station, track	product

Event	Trigger	Key properties	Lattice
gate_stalled	14d no sessions	hint	product
station_reached	Any station EARNED	station, track, door	product
door_opened	Door entered	door, epoch	product
register_filter_used	Filter applied	register, surface	product
tag_contested	Contest filed	folio, from, to	product
pratibimba_viewed	Mirror page opened	page	product
yantra_fallback_shown	Template/generic served	source, band	product

Table 12.1 — Event dictionary. Business lattice per D03 Part VII; product lattice new in this document (G-09).

12.2 Envelope and governance

LEAD-EVENT ENVELOPE (TYPESCRIPT COMMENT)

```
// every LeadEvent row
{ event, path, referrer, props, at }
// referrer classes: ai_referral | karma | organic | direct | repeat
// privacy classes:
//   P0 anonymous -> no PII, safe to aggregate publicly
//   P1 seeker-scoped -> joins Seeker, never exported raw
// retention: 26 months, then aggregate-only (counts by week)
```

- The six business events are one line of instrumentation each and join to source by referral path — D03’s requirement, restated as a schema constraint.
- Gate events make the retention model measurable: the `gate_evaluated` → `gate_crossed` funnel is the instrument that will confirm or refute §8’s hybrid model with real seekers. If crossing rates do not improve against the strict baseline, the decision reopens (D-01 records the condition).
- `yantra_fallback_shown` is the engine’s health metric: a rising fallback rate means provider degradation, questionnaire drift, or weight-matrix error — each diagnosable from YantraRun audit rows.
- PRATIBIMBA events carry no profile inference. The platform that documents the anatomy of deception does not run it on its own seekers (D03 VII).

13. Security: The Access-File Finding

The specification package arrived with a companion PDF referred to in Dossier No. 03 as “the credentials file: the deployment tokens that made it possible to pull the live deployment record rather than guess at it.” That file contains live production secrets in plaintext: a GitHub personal access token for the platform’s mirror repository, Cloudinary API credentials, an OpenRouter API key, the Turso database URL with its auth token, a Vercel deployment token, and the admin login for the production site — the last with its password literally set to a placeholder. By being attached here, every one of those secrets must now be considered compromised. No secret is reproduced in this document, and none needs to be: the finding is about exposure, not values.

6 systems

credentials exposed by transmission — all require rotation

0 excuses

rotation precedes every other item in the roadmap

pre-existing

D03 Part IX confirms live use of these tokens in August 2026

13.1 Rotation runbook (P0 — today, before any code)

System	Action	After rotation
GitHub PAT	Revoke; issue a fine-grained token scoped to the single repo, least privileges.	Stored only in the platform vault.
Cloudinary	Rotate API secret from the dashboard.	Env vars via Vercel encrypted settings.
OpenRouter	Issue new key; set a monthly spend cap.	Repo secret for Actions; runtime env for the app.
Turso	Rotate the database auth token; rotate the DB password.	Never again in any document or chat.
Vercel	Revoke and reissue the deployment token.	CI-only scope.
Admin login	Replace the placeholder password; enable 2FA.	Rate-limited login; audit log on.

Table 13.1 — Rotation runbook. Urgency for every row: immediately.

13.2 Standing policy

- **Secrets live in exactly one place:** the platform’s encrypted environment store (Vercel) and repository secrets for CI. NEVER in documents, chat transcripts, emails, or filenames. MUST.
- **Documents need tokens only for the duration of a task:** issue scoped, expiring credentials; revoke on completion. The engagement that produced D03 needed read-only deployment access, not a mutable PAT. SHOULD.
- **Secret scanning in CI:** gitleaks (or equivalent) on every push, aligned with the D03 doctrine that the monitor catches what hands forget. MUST.
- **Quarterly audit:** credential age, scope, and last-rotation date reviewed on the same calendar as the content refresh cycle (§11.2). SHOULD.

- **The access file is deleted** from every storage location after rotation is confirmed, including backups and this engagement’s working directory. **MUST.**

One structural note in the platform’s favour: the architecture already runs its intelligence server-side, so rotating the OpenRouter key and the Turso token breaks nothing in the seeker-facing product. The exposure is an operational wound, not an architectural one — which is precisely why it must be closed before the build makes the surface larger.

14. Deferred Concerns

Three of the review’s missing sections are real but not load-bearing for v1. Each gets a position now so that deferral is a decision with a trigger, not an omission that becomes a discovery mid-build.

14.1 Internationalization: IAST is the product

The platform’s Sanskrit is IAST-transliterated, and that is a stance, not a gap: the entire anti-middleman doctrine rests on making precise vocabulary readable without a priest’s pronunciation. Hindi and other Indic-language surfaces are a genuine audience question — but a post-revenue one. What v1 owes the future is structure: correct `lang` attributes, Devanagari renderings beside IAST in `LexiconTerm` rows as a display enhancement, and translation-unfriendly copy kept out of components. UI localization opens when the karma cluster’s numbers justify it; the trigger is recorded, the decision is D-eligible.

14.2 Offline: one surface, done properly

Sādhana happens in ashrams and retreats with poor connectivity, and the one surface that must work there is the practice logger — the gate’s fuel. The spec: an offline-first logger (service worker + `IndexedDB` queue, idempotent upserts keyed on seeker-local date, visible pending state, background sync on reconnect, conflicts resolved in favour of the earliest local timestamp). The rest of the platform does not need PWA treatment; a Dossier generated in a cave is a nice story but a false requirement. The logger ships in Phase 3 with the gate it feeds.

14.3 Accessibility: WCAG 2.2 AA as the floor

- Contrast: register chips and severity colours audited against AA — the palette’s low-saturation discipline makes this cheap if checked at component birth, expensive if retrofitted. **MUST** from Phase 1.
- Screen readers: IAST terms carry `aria-label` with a plain-romanization pronunciation hint; the Dossier flow is fully keyboard-navigable; progress-map state is exposed as text, never colour alone. **MUST.**
- Motion: all transitions respect `prefers-reduced-motion`; the progress map’s orbit animation degrades to a static diagram. **MUST.**

15. Phased Build Roadmap

Five phases, each with an acceptance test, sequenced so that nothing builds on an unverified base — the same doctrine D03 applies to its own calendar. The phases absorb D03's where they overlap: Phase 0 here is D03 Phase 0 plus the security closure; Phase 4 aligns with D03 Phase 3's defense machine. Durations assume one developer.

Phase · ships	Depends on	Acceptance test	Risk if skipped
0 • Foundation (week 0): secrets rotated; DECISIONS.md seeded; constants file; golden-set repo skeleton.	Nothing — precedes all code.	Rotation confirmed per system; constants imported (never typed) in a hello-world build.	Every later phase inherits exposed keys and drifting counts.
1 • Ledger + accounts (wks 1–2): schema migration; Seeker accounts; folios render with register tags + filters.	Phase 0.	Seed counts assert 56/20/86/16; register filter fires events; a lly audit of the chip set passes AA.	The taxonomy ships as decoration instead of machinery (S-01 obligation).
2 • The Mirror (wks 3–5): YANTRA core + gate; Dossier generation, persistence, versions; guru export; YantraRun audit.	Phase 1.	Golden set green (30 fixtures); determinism reproduced across two runs; template narrative < 2s on provider outage.	The differentiating engine ships unaudited — the core IP risk.
3 • The Spine (wks 6–8): offline logger; hybrid gate; stations render; badges; preview register.	Phase 2.	21-in-30 evaluation deterministic; offline log syncs without double-count; gate events flowing.	Retention cliff survives into production; G-03 becomes churn data.
4 • The Field + ops (wks 9–12): doors + Echo tracks; case files; Observatory; analytics lattices; kalki-geo-ops deploy (D03).	Phase 3.	Two tracks render on one map; contest lifecycle exercised end-to-end; both lattices read in one half-hour monthly.	The ring/door promise and the Field remain prose.

Table 15.1 — Build phases with acceptance tests. One developer assumed; durations double for two, not halve.

The dependency that matters most is invisible in the table: the data model (§6) is Phase 1's first artifact, and every later phase consumes it. The reviewer's recommendation — the schema is the contract, produced before any code — is enforced by making migration #1 the acceptance test of Phase 1. Similarly, DECISIONS.md opens in Phase 0 with this document's six open decisions already in it, so the reconciliation habit starts before the first disagreement.

16. Register of Open Decisions

Six decisions belong to the owner, not the developer. Each is recorded in the spec’s own reconciliation format so that resolving it is a one-line edit to DECISIONS.md, and each carries the consequence of leaving it open — because an unmade decision is also a decision, made by whoever writes the code first.

ID	Decision	Recommendation	Cost of deferral
D-01	Gate model: strict 21-consecutive vs rolling 21-in-30 vs hybrid (§8).	Hybrid — window + grace tokens + PARĪKṢā preview.	Phase 3 builds the retention cliff into schema and copy; reversal costs a migration.
D-02	YANTRA provider posture: OpenRouter-only vs template-fallback posture (§5).	OpenRouter primary + template fallback , temperature 0, cached narratives.	Either an outage risk or a narrative-quality cliff, decided by accident.
D-03	Tag adjudicator: solo archivist vs committee (§9.4).	Archivist now ; committee is a delegation later.	The 14-day SLA has no owner; contests queue silently.
D-04	IP protection for the four-register taxonomy (§4.1).	Trademark filing this quarter ; patent exploration as a slower parallel track.	The strongest defensible asset remains free for competitors to name-adopt.
D-05	Door-scoped stations: parallel tracks vs door overlays (§7).	Parallel tracks — both truths tracked.	The Echo Rule ships as copy instead of state; progress maps guess.
D-06	Guru export format: PDF vs share-link (§4.2).	PDF at Phase 2 ; share-link (expiring, seeker-controlled) later.	The between-seeker-and-teacher positioning waits on a widget.

Table 16.1 — Open decisions. Status for all six: OPEN.

Resolve D-01 and D-04 first: one shapes the schema, the other protects the asset the schema serves. The remaining four can be resolved in the week their phase begins, provided DECISIONS.md is alive from Phase 0 to receive them. The pattern is the spec’s own: conflicts become records, records become the documentation, and the next argument is answered by pointing at a line.